

ФГАОУ ВО «НИУ ИТМО»
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Лабораторная работа №3

Решение систем нелинейных уравнений

(по дисциплине «Вычислительная математика»)

Выполнил:

Лабин Макар Андреевич,
ВЫЧМ 3.2, Р3231, 466449.

Проверила:

Перл Ольга Вячеславовна

ИТМО

г. Санкт-Петербург, Россия
2026

Содержание

1	Постановка задач	1
2	Выполнение работы	2
2.1	Описание метода	2
2.1.1	Начальные данные	2
2.1.2	Условия сходимости	2
2.1.3	Критерий останова и его обоснование	3
2.2	Блок-схема численного метода	3
2.3	Листинг численного метода	5
2.4	Результаты работы программы	7
2.5	Результаты численного моделирования	8
3	Заключение	9

1 Постановка задач

Дана система нелинейных уравнений вида $F(X) = 0$. По заданному начальному приближению $X^{(0)}$ необходимо найти решение системы с точностью до 5-го верного знака после запятой.

В варианте №2 решение осуществляется *методом простых итераций*.

То есть в ходе лабораторной работы будут выполнены следующие задачи:

1. Приведение исходной системы нелинейных уравнений к итерационному виду $X = \Phi(X)$, который подходит для применения метода.
2. Разработка алгоритма, который реализует метод простых итераций для системы уравнений.
3. Вычисление вектора корней системы по заданным начальным приближениям и выбранному номеру системы.

2 Выполнение работы

2.1 Описание метода

2.1.1 Начальные данные

Рассмотрим систему нелинейных уравнений размерности n в векторном виде:

$$\vec{F}(\vec{x}) = \vec{0}, \quad \vec{F}(\vec{x}) = (f_1(x_1, \dots, x_n), \dots, f_n(x_1, \dots, x_n))^T.$$

Для применения метода простых итераций исходную систему необходимо преобразовать к эквивалентному виду

$$\vec{x} = \Phi(\vec{x}),$$

где $\Phi : \mathbb{R}^n \rightarrow \mathbb{R}^n$ — некоторая вектор-функция. Такое преобразование не является единственным. Сходимость метода напрямую зависит от выбора Φ .

2.1.2 Условия сходимости

Для сходимости итерационного процесса $\vec{x}^{(k+1)} = \Phi(\vec{x}^{(k)})$ исследуем невязку:

$$\vec{e}^{(k)} = \vec{x}^{(k)} - \vec{x}^*$$

Рассмотрим класс функций, дифференцируемых в точке $\vec{x}^*$. Разложим $\Phi(\vec{x})$ в ряд Тейлора в окрестности $\vec{x}^*$:

$$\Phi(\vec{x}^{(k)}) = \Phi(\vec{x}^*) + J_\Phi(\vec{x}^*)(\vec{x}^{(k)} - \vec{x}^*) + O(\|\vec{x}^{(k)} - \vec{x}^*\|^2)$$

где $J_\Phi(\vec{x}^*)$ — матрица Якоби. Учтём, что $\vec{x}^{(k+1)} = \Phi(\vec{x}^{(k)})$ и $\vec{x}^* = \Phi(\vec{x}^*)$:

$$\begin{aligned} \vec{x}^{(k+1)} &\approx \vec{x}^* + J_\Phi(\vec{x}^*)(\vec{x}^{(k)} - \vec{x}^*) \\ \vec{e}^{(k+1)} &\approx J_\Phi(\vec{x}^*)\vec{e}^{(k)} \end{aligned}$$

То есть условие сходимости сводится к требованию, чтобы спектральный радиус матрицы Якоби в точке решения был меньше единицы:

$$q \equiv \rho(J_\Phi(\vec{x}^*)) < 1$$

Из этого можем вывести следствие: если в некоторой окрестности решения существует такая согласованная матричная норма, что:

$$\|J_\Phi(\vec{x})\| \leq q < 1,$$

то итерационная последовательность сходится к решению $\vec{x}^*$ *линейно*:

$$\|\vec{e}^{(k+1)}\| \leq q \|\vec{e}^{(k)}\|$$

2.1.3 Критерий останова и его обоснование

Для контроля точности итерационного процесса необходимо установить связь между расстоянием до точного решения $\|\vec{x}^{(k)} - \vec{x}^*\|$ и разностью двух последовательных приближений $\|\vec{x}^{(k)} - \vec{x}^{(k-1)}\|$.

Оценим расстояние от текущей итерации до корня как сумму всех последующих шагов через неравенство треугольника:

$$\|\vec{x}^{(k)} - \vec{x}^*\| = \left\| \sum_{i=k}^{\infty} (\vec{x}^{(i)} - \vec{x}^{(i+1)}) \right\| \leq \sum_{i=k}^{\infty} \|\vec{x}^{(i)} - \vec{x}^{(i+1)}\|$$

Учтём, что каждый шаг уменьшается пропорционально q :

$$\|\vec{x}^{(i)} - \vec{x}^{(i+1)}\| \leq q^{i-k} \|\vec{x}^{(k)} - \vec{x}^{(k+1)}\|$$

Подставляя это в сумму, получим убывающую геометрическую прогрессию:

$$\|\vec{x}^{(k)} - \vec{x}^*\| \leq \|\vec{x}^{(k)} - \vec{x}^{(k+1)}\| \cdot (1 + q + q^2 + \dots) = \frac{1}{1-q} \|\vec{x}^{(k)} - \vec{x}^{(k+1)}\|$$

Воспользуемся поправкой на один шаг и получим финальную оценку:

$$\|\vec{x}^{(k)} - \vec{x}^*\| \leq \frac{q}{1-q} \|\vec{x}^{(k)} - \vec{x}^{(k-1)}\|$$

Таким образом, для достижения заданной точности ε расчёт необходимо вести до выполнения условия:

$$\|\vec{x}^{(k)} - \vec{x}^*\| \leq \varepsilon \implies \|\vec{x}^{(k)} - \vec{x}^{(k-1)}\| \leq \frac{1-q}{q} \varepsilon$$

Если метод сходится быстро, то коэффициент перед погрешностью в финальной оценке становится меньше или равен единице:

$$q \leq 0.5 \implies \frac{q}{1-q} \leq 1.$$

В этом случае условие $\|\vec{x}^{(k)} - \vec{x}^{(k-1)}\| \leq \varepsilon$ гарантирует, что истинная погрешность $\|\vec{x}^{(k)} - \vec{x}^*\|$ также не превышает ε .

2.2 Блок-схема численного метода

На вход реализуемого метода подаются следующие данные:

- n — размерность системы; является целым числом.
- A — расширенная матрица системы; является двумерным массивом вещественных чисел двойной точности.

На выходе метода ожидается вектор над вещественными числами двойной точности — решение системы, которое совмещено с таким же вектором невязки для данного решения.

В случае, если система не имеет единственного решения, метод должен выбросить соответствующую ошибку.

Далее приводятся блок-схемы, реализующие численный метод.

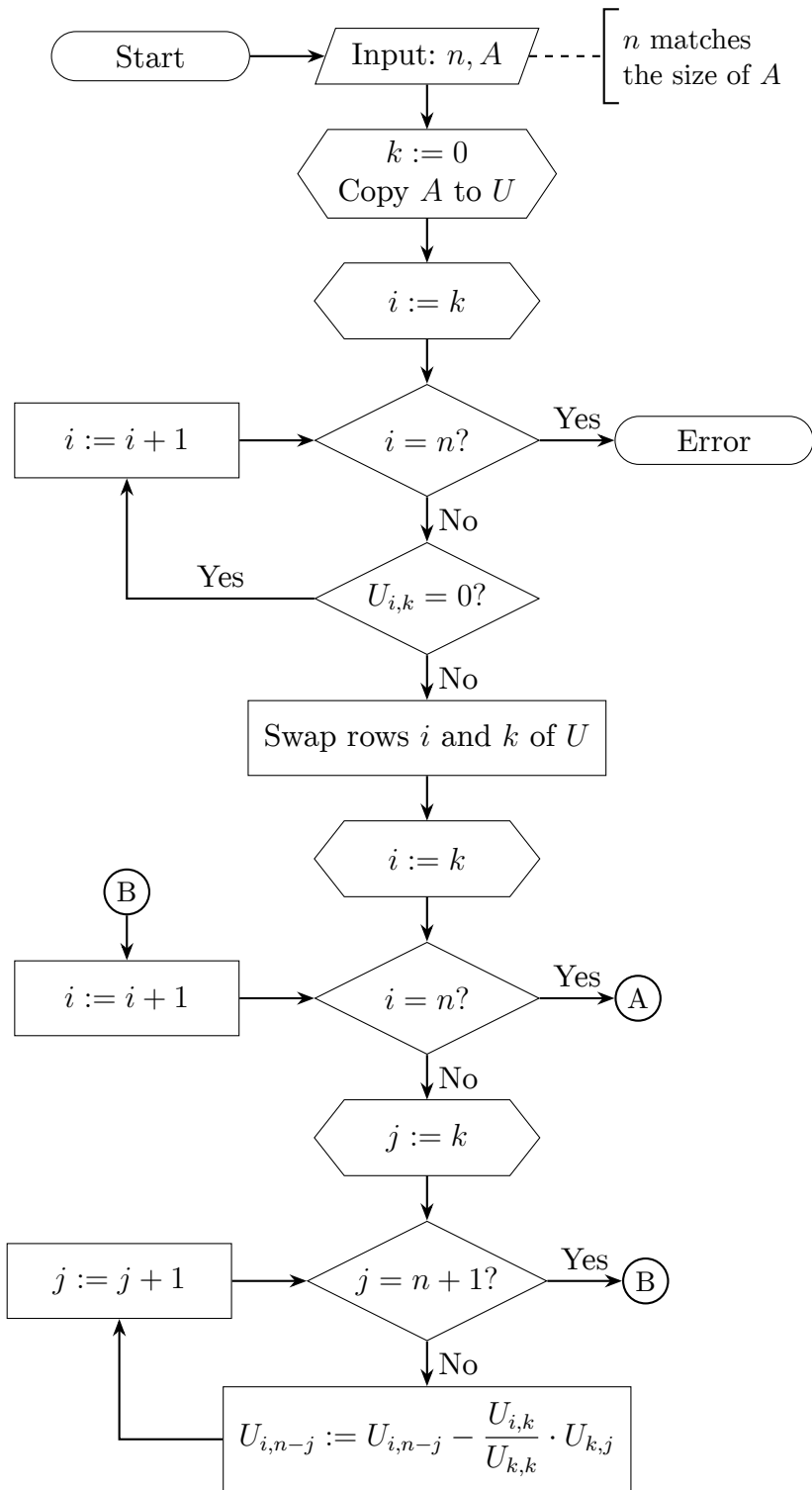


Схема 1 — Прямой ход метода Гаусса.

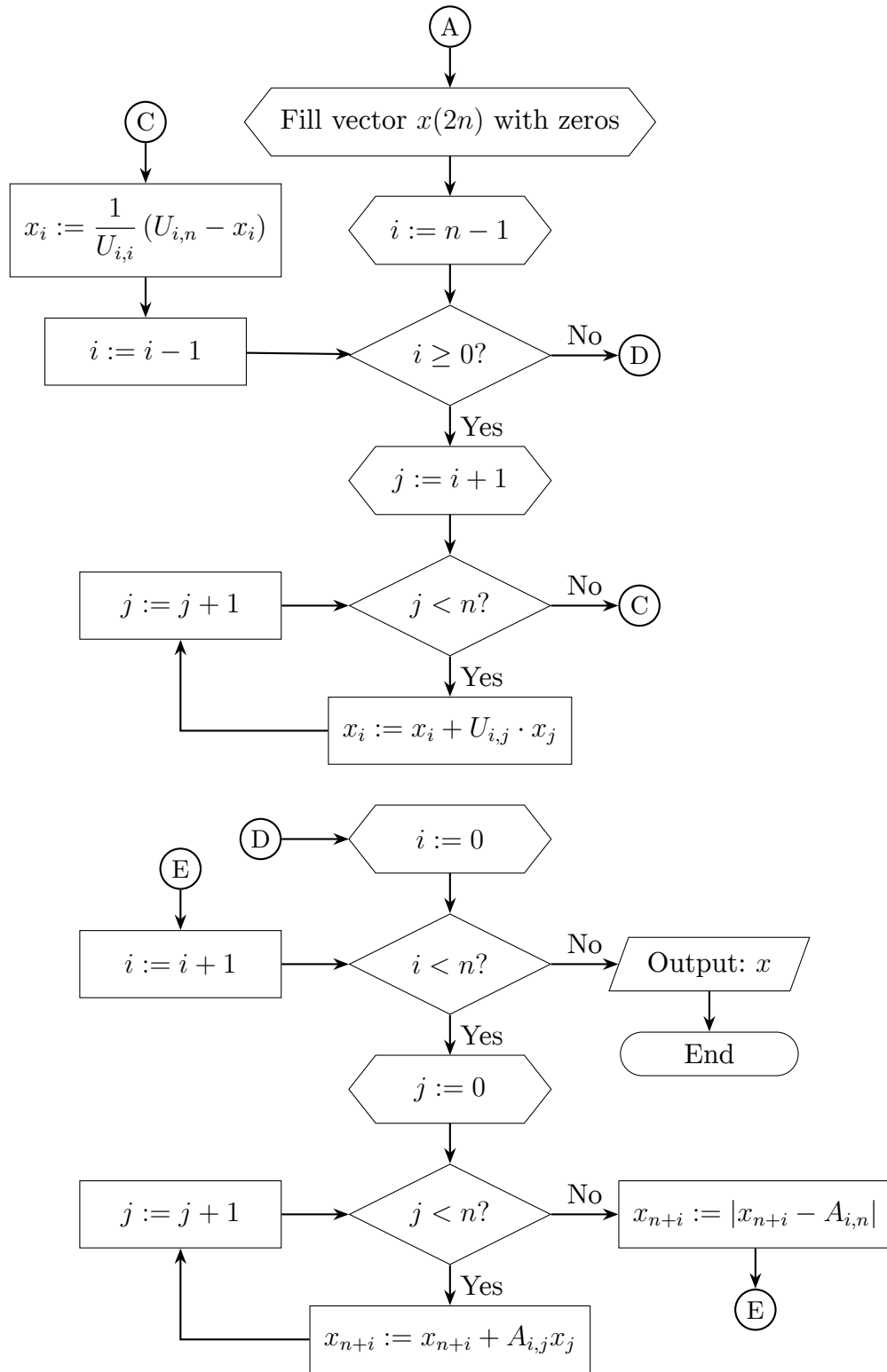


Схема 2 — Обратный ход метода Гаусса.

2.3 Листинг численного метода

Для реализации блок-схем воспользуемся языком программирования C++.

```

1 bool triangulateMatrix(const int n, vector<vector<double>>&
  matrix) {
2   bool isSingular = false;

```

```

3  for (size_t i = 0; i < n - 1; i++) {
4      size_t idx = n;
5      for (size_t j = i; j < n; j++) {
6          if (matrix[j][i] != 0) {
7              idx = j;
8              break;
9          }
10     }
11     if (idx == n) {
12         isSingular = true;
13         continue;
14     }
15     // apply P operator
16     if (idx != i) {
17         vector<double> temp_row = matrix[idx];
18         matrix[idx] = matrix[i];
19         matrix[i] = temp_row;
20     }
21     // apply M operator
22     for (size_t j = i + 1; j < n; j++) {
23         double factor = matrix[j][i] / matrix[i][i];
24         for (size_t k = i; k <= n; k++) {
25             matrix[j][k] -= factor * matrix[i][k];
26         }
27     }
28 }
29 if (isSingular) {
30     errorMessage = "The system has no single solution.";
31     isSolutionExists = !hasContradiction(n, matrix);
32     return false;
33 }
34 return true;
35 }

```

Листинг 1 — Прямой шаг метода Гаусса.

```

1  bool hasContradiction(const int n, vector<vector<double>>&
2      matrix) {
3      vector<double> zeroes(n, 0);
4      for (size_t i = 0; i < n; i++) {
5          vector<double> curr_row = matrix[i];
6          curr_row.pop_back();
7          if (curr_row == zeroes && matrix[i][n] != 0) {
8              return true;
9          }
10     }
11     return false;
12 }

```

Листинг 2 — Проверка системы на совместность.

```

1  vector<double> solveByGauss(int n, vector<vector<double>> matrix
2      ) {

```



```

2  vector<double> answer(2 * n, 0);
3  vector<vector<double>> matrix_triangle = matrix;
4  if (!triangulateMatrix(n, matrix_triangle)) {
5      return answer;
6  }
7  for (size_t i = 0; i < n; i++) {
8      const size_t idx = n - i - 1;
9      for (size_t j = idx + 1; j < n; j++) {
10         answer[idx] -= matrix_triangle[idx][j] * answer[j];
11     }
12     answer[idx] += matrix_triangle[idx][n];
13     answer[idx] /= matrix_triangle[idx][idx];
14 }
15 // calculate residuals
16 for (size_t i = 0; i < n; i++) {
17     for (size_t j = 0; j < n; j++) {
18         answer[n + i] += matrix[i][j] * answer[j];
19     }
20     answer[n + i] -= matrix[i][n];
21 }
22 return answer;
23 }

```

Листинг 3 — Обратный шаг метода Гаусса.

2.4 Результаты работы программы

Для демонстрации работы метода и тестирования программы рассмотрим пять систем различной природы.

1. *Тригонометрическая система:*

$$\begin{cases} \sin(x_1) = 0 \\ \frac{x_1 x_2}{2} = 0 \end{cases}$$

2. *Алгебраическая система высоких степеней:*

$$\begin{cases} x_1^2 x_2^2 - 3x_1^3 - 6x_2^3 + 8 = 0 \\ x_1^4 - 9x_2 + 2 = 0 \end{cases}$$

3. *Алгебраическая система:*

$$\begin{cases} x_1 + x_1^2 - 2x_2 x_3 - 0.1 = 0 \\ x_2 + x_2^2 + 3x_1 x_3 + 0.2 = 0 \\ x_3 + x_3^2 + 2x_1 x_2 - 0.3 = 0 \end{cases}$$

4. *Смешанная алгебраическая система:*

$$\begin{cases} x_1 - \sqrt{1 - x_2^2} = 0 \\ x_2 - x_1^3 = 0 \end{cases}, \quad |x_2| \leq 1$$

5. Трансцендентная система:

$$\begin{cases} x_1 + e^{x_1} - 2 = 0 \\ x_2 + \sin(x_2) - x_1 = 0 \end{cases}$$

2.5 Результаты численного моделирования

Ниже приведены результаты решения указанных систем методом простых итераций с заданной точностью $\epsilon = 10^{-5}$.

ID системы	Начальное приближение $\vec{x}^{(0)}$	Решение $\vec{x}^*$	Кол-во итераций
1	(0.5, 0.5)	(0.0000, 0.0000)	8
2	(1.0, 0.5)	(1.1231, 0.3982)	15
4	(0.8, 0.5)	(0.8261, 0.5638)	12
5	(0.5, 0.5)	(0.4428, 0.2241)	10

Таблица 1 — Сводная таблица результатов тестирования

3 Заключение

После выполнения лабораторной работы у меня появились ответы на следующие вопросы.

Почему используемый критерий останова может быть неточным?

В работе мы допустили, что метод сходится быстро. Если же сходимость «медленная» ($q \rightarrow 1$), то разность двух соседних приближений может быть очень малой, в то время как само решение всё еще находится далеко от текущей точки. В таких ситуациях использование упрощенного критерия без учета q может привести к преждевременной остановке алгоритма до достижения реальной точности.

Почему у матриц $A_1, \dots, A_7$ возникла ошибка? Матрицы $A_1, \dots, A_3$ обладают общим свойством: их определитель равен нулю. Метод Гаусса не может использоваться для вырожденных матриц, поскольку спроектирован для поиска единственного решения. В нашем случае решений нет: для любых векторов система противоречива из-за ненулевого $\vec{f}$.

Матрицы $A_4, \dots, A_6$ внешне ничем не примечательны, но можно заметить, что их строки линейно зависимы. Это тоже означает, что решение не может быть единственным. После приведения расширенных матриц к верхнетреугольному виду выяснится, что в системах также есть противоречие, и решений нет.

Наконец, матрица A_7 имеет нулевой элемент на главной диагонали. Более того, на этой же строке имеется нулевой элемент у $\vec{f}$. Это значит, что x_4 — свободная переменная, и решений у системы бесконечно много.

Какая асимптотическая сложность у алгоритма? Временная сложность метода Гаусса составляет $\mathcal{O}(n^3)$. Как следует из блок-схемы, прямой ход содержит три вложенных цикла (по переменным i, j, k). Обратный ход содержит два вложенных цикла, потому имеет сложность $\mathcal{O}(n^2)$. Пространственную сложность можно оценить $\mathcal{O}(n^2)$, поскольку алгоритм требует хранения всей матрицы.

Почему бы не воспользоваться методом Крамера? По сравнению с обычным методом Крамера, метод Гаусса эффективнее для систем большой размерности. Метод Крамера требует вычисления $n + 1$ определителя, что является асимптотически сложной вычислительной операцией, а конкретно — $\mathcal{O}(n^4)$.

Почему невязка везде нулевая? Предположительно это связано с видом рассматриваемых матриц. Как можно заметить, в отчёте приведены матрицы с относительно небольшими по модулю коэффициентами и малой дробной частью (либо вообще целые числа).

Если бы была дана матрица с коэффициентами, у которых мантисса велика, то точность вычисляемых значений упала бы из-за ограниченности разрядной сетки типа `double`. Для нашей реализации метода это было бы особен-

но заметно, потому что все операции последовательные, и ошибка бы только накапливалась, что привело бы к ненулевой невязке.